

B2: Spaces

The space is the boundary the whole Permissioned Data proposal is built around. Everything permissioned lives inside one: records are stored per space, authorized per space, and synced per space. This lesson covers the three values that identify a space, how spaces and permissioned records get **at://** URIs, how a space authority resolves to key material and a host, and how space types are declared in Lexicon.

The (authority, type, skey) triple

What a space is

A **space** is an authorization and sync boundary representing a shared social context. A space may include many different types of records from many users. The space does not colocate records on a single network host. Instead, each user stores their own records for a given space in a permissioned repo on their own repo host. A space is the aggregation of these per-user repos across the network: an application presenting a space pulls each member's repo from its host, assembles the view, and applies access control to requesting users.

Permissioned Data proposal, "Spaces"

So a space is a boundary, not a storage location. There is no single server where the space lives. Each participating user keeps their own records for the space in a **permissioned repo** on their own repo host, typically their PDS. The space only exists as the aggregation of those per-user repos: an application pulls each member's repo from its host, assembles the combined view, and enforces access control on whoever is asking.

That also changes the repo math. Public broadcast gives you one repo per user. Permissioned data gives you one permissioned repo per (user, space) pair, so a user participating in three spaces has three separate permissioned repos.

The three identifying values

Each space is identified by three values:

1. **space authority**: a DID, the root of authority for the space
2. **space type**: an NSID describing the modality of the space
3. **space key** (**skey**): a string distinguishing spaces of the same space type under the same space authority

Each element of the triple has a different syntactic form, which makes them easy to tell apart in a URI:

Element	Form	Role
Space authority	DID	Root of authority for the space; issuer of its credentials
Space type	NSID	Names the space's modality (forum, chat, bookmarks, ...)
Space key (skey)	string	Distinguishes spaces of the same type under the same authority

Nothing in the triple is a hosting address, a member list, or a credential. Hosts are resolved separately through the authority's DID document (covered below), and access is granted separately through space credentials:

Reading or syncing a space requires a **space credential** signed by the declared signing key of the space authority. The space authority decides whether to issue one based on the requesting user and client application. The protocol does not define how that decision is made and carries no member list (see Access Control). Spaces scale from a single user's personal data (e.g. bookmarks) to communities of millions of users.

The space key (skey)

The **space key** (**skey**) is a string distinguishing spaces of the same type under the same authority, analogous to a record key (**rkey**): a slug, a TID, a cryptographic identifier, or a reserved string such as **self**. Maximum length 512 bytes with the same syntax requirements as an **rkey**.

If you know rkeys, you know skeys: same syntax rules, capped at 512 bytes, and no single form mandated. A slug, a TID, a cryptographic identifier, or a reserved string like **self** all work. The uniqueness obligation is narrow. An skey only has to distinguish spaces of the same type under the same authority; global uniqueness comes from the full (authority, type, skey) triple. So the same skey can appear under different authorities, or under different types, without any collision.

Addressing & URIs

The permissioned record URI

A permissioned record is addressed by an **at://** URI with a fixed **space** segment:

at://{spaceDid}/space/{spaceType}/{skey}/{authorDid}/{collection}/{rkey}

The proposal defines each component:

Component	Type	Description
spaceDid	DID	Space authority DID
space	literal	Fixed marker identifying this as a permissioned space URI
spaceType	NSID	Space type
skey	string	Space key
authorDid	DID	DID of the record's author
collection	NSID	Record collection
rkey	string	Record key

A full record URI carries two DIDs doing different jobs. The first, `spaceDid`, is the space authority, and it holds the URI authority position. The second, `authorDid`, comes after the skey and names the user who wrote the record. This split is deliberate: record authority still rests with the author's user DID, as in public broadcast, but URI authority moves to the space authority DID. A concrete URI for a thread record written by Alice in a "gardening" forum:

at://did:plc:forum123/space/com.atmoboards.forum/gardening/did:plc:alice42/com.atmoba

Here `did:plc:forum123` is the space authority, `com.atmoboards.forum` the space type, `gardening` the skey, `did:plc:alice42` the record author, `com.atmoboards.thread` the collection, and `3jt5exlkr2c2v` the rkey. No hosting information appears anywhere in the URI. Hosts are resolved from DID documents, never encoded in addresses.

Why permissioned and public URIs never collide

Permissioned data reuses the `at://` scheme rather than defining its own. The literal `space` segment sits where a collection NSID appears in a public atproto URI, so a permissioned URI is distinguished from a public one by that marker in the first path segment under the authority DID. The two are never ambiguous, as a public collection is an NSID, which always contains at least two `.`s, whereas the `space` marker contains none.

Permissioned Data proposal, "Addressing"

A parser looking at `at://did:plc:xyz/first-segment/...` decides on the first path segment alone. The literal `space` has zero dots, so the URI is permissioned. An NSID always has at least two dots, so the URI is a public record URI. That's the whole trick: no separate scheme, no query parameter, no reserved DID method, just a fixed path marker that can never look like an NSID.

Referencing a space vs. a record

All segments through **rkey** are necessary to identify a **permissioned record**. The leading segments through **skey** may be used to reference a **space**:

Permissioned Data proposal, "Addressing"

Space: `at://{spaceDid}/space/{spaceType}/{skey}`

Record: `at://{spaceDid}/space/{spaceType}/{skey}/{authorDid}/{collection}/{rkey}`

To reference the gardening forum itself, truncate after the skey:

`at://did:plc:forum123/space/com.atmboards.forum/gardening`

Watch the segment order: the space type NSID always comes before the skey, mirroring the order of the identity triple.

Space authority resolution

What the authority is

A space's **authority** is the DID at the root of the space and the issuer of its credentials. It may be a user's own DID as for personal data such as bookmarks or mutes. Or it may be a dedicated DID which lets a shared space transfer between users independently of any individual account.

Permissioned Data proposal, "Space authority"

The authority is an ordinary DID. For personal spaces like bookmarks or mutes it's simply the user's own DID. For shared spaces a dedicated DID buys you something concrete: the space's identity hangs off that DID rather than any person's account, so the space can change hands without changing its identity or breaking a single URI.

The two DID document entries

A space authority is resolved through two entries in its DID document:

- a **verification method** with id **#atproto_space**: the public key used to verify the space's credentials
- a **service** entry with id **#atproto_space_host**: the endpoint of the space host

Permissioned Data proposal, "Space authority"

The two entries answer different questions, and quiz questions love to swap them:

Entry id	DID document kind	Answers
#atproto_space	Verification method (a public key)	Which key verifies the space's credentials?

Entry id	DID document kind	Answers
<code>#atproto_space_host</code>	Service entry (an endpoint)	Which service answers for the space as a whole?

Fallback behavior

Both entries are optional. When `#atproto_space` is absent, the space signing key falls back to the account's `#atproto` signing key. Similarly, when `#atproto_space_host` is absent, the space host falls back to the account's `#atproto_pds` service endpoint. An authority MAY instead publish the dedicated entries to point at distinct key material or a distinct host, and MAY set them to the same values as `#atproto` and `#atproto_pds` explicitly.

Permissioned Data proposal, "Space authority"

The fallbacks pair up cleanly, and a missing entry is never an error. Resolution always succeeds against an ordinary atproto account's DID document:

Dedicated entry	Fallback when absent
<code>#atproto_space</code> (space signing key)	Account's <code>#atproto</code> signing key
<code>#atproto_space_host</code> (space host endpoint)	Account's <code>#atproto_pds</code> service endpoint

A missing entry never weakens the access model. Reading or syncing a space still requires a space credential signed by the authority's declared signing key. The fallback only changes which key that is (the account's regular `#atproto` key) and which host serves the space (the account's PDS). This is also why a PDS covers both roles by default: the proposal's terminology section describes a PDS acting as both a **repo host**, storing and serving users' permissioned repos, and a **space host**, answering for a space as a whole by issuing credentials, enumerating writers, and routing notifications. Either role may instead be filled by any service implementing the required APIs.

Space type declarations

The space type

A space's **type** is an NSID that names its modality and resolves to a space type declaration. It identifies the kind of data a space holds before any network resolution, much as a collection NSID does in public atproto. Because a type names a concrete modality, every space is some specific kind of space rather than a generic container.

Permissioned Data proposal, "Space type"

The type is also the **OAuth consent boundary**. Access is granted to a user by type, e.g. "access to your AtmoBoards forums" (see OAuth scopes).

Permissioned Data proposal, "Space type"

So the type does two jobs. It tells you what kind of space you're looking at before you resolve anything on the network, and it sets the granularity at which users consent to application access. Users grant access to a type of space ("your AtmoBoards forums"), not to individual collections inside it.

The declaration document

A space type NSID resolves to a **space type declaration**: a Lexicon definition with **"type": "space"**.

Permissioned Data proposal, "Space type declarations"

```
{
  "lexicon": 1,
  "id": "com.atmoboards.forum",
  "defs": {
    "main": {
      "type": "space",
      "description": "A discussion forum",
      "key": "any",
      "name": "AtmoBoards Forum",
      "name:lang": { "es": "Foro AtmoBoards", "ja": "AtmoBoards" },
      "collections": ["com.atmoboards.thread", "com.atmoboards.reply"]
    }
  }
}
```

The proposal specifies the fields as follows:

Field	Type	Required	Description
type	"space"	yes	Marks this as a space type declaration. Must be the main definition.
description	string	no	Description of the space type for developers. Not shown to users.
key	string	yes	Specifies the recommended space key type (similar to record key types)
name	string (1–64)	yes	Human-readable name for the space type, shown to users on consent screens.
name:lang	map<lang, string>	no	Localized name values by language code.
collections	array of NSID	yes	Collections clients should expect in a space of this type.

The **"type": "space"** marker must sit on the **main** definition of the Lexicon document. Four fields are required (**type**, **key**, **name**, **collections**) and two are optional (**description** and **name:lang**). The

user-facing one is **name**: 1 to 64 characters, shown on consent screens. **description** is for developers and is explicitly not shown to users, so don't put consent language there. **key** recommends a space key type for spaces of this type, analogous to record key types ("**any**" in the example above).

What the collections field does

The **collections** field is the default **collection** set for a **space** scope of this type. However, ultimately any collection may be written to any space and is not constrained at the protocol level by the collections in a space type declaration.

Permissioned Data proposal, "Space type declarations"

So **collections** is advisory plus a scope default. It tells clients what record collections to expect in a space of this type, and it seeds the default **collection** set when an application requests a **space** OAuth scope for the type. It is not a protocol-level write restriction: any collection may be written to any space, whatever the declaration lists.

You should now be able to explain

- What a space is: an authorization and sync boundary, aggregated from per-user permissioned repos rather than colocated on any single host
- The (authority, type, skey) identity triple and the syntactic form of each element (DID, NSID, string)
- The rules for space keys: rkey-like syntax, 512-byte maximum, and uniqueness only among same-type spaces under the same authority
- The full permissioned record URI pattern **at://{spaceDid}/space/{spaceType}/{skey}/{authorDid}/{collection}/{rkey}**, the meaning of every segment, and the two distinct DIDs it carries
- How the literal **space** segment distinguishes permissioned URIs from public ones (zero dots vs. an NSID's two-plus dots)
- How to reference a space itself by truncating the URI after the skey
- What the space authority is, why a dedicated DID lets a shared space transfer between users, and that a user's own DID can serve for personal spaces
- The two DID document entries, **#atproto_space** (verification method, the credential key) and **#atproto_space_host** (service entry, the host endpoint), and their fallbacks to **#atproto** and **#atproto_pds**
- That reading or syncing a space always requires a space credential signed by the authority's declared signing key
- The structure of a space type declaration: a Lexicon **main** definition with "**type**": "**space**", its required and optional fields, and which field appears on consent screens

- Why the space type is the OAuth consent boundary, and why **collections** is a scope default rather than a protocol-level write constraint